

Prova di Continuità: un modello temporale per la propagazione dell'autorità nei sistemi distribuiti e negli agenti di IA

Nicola Gallo

8 luglio 2026

Sommario

I modelli di autorizzazione basati sulla Prova di Possesso fanno discendere l'autorità dal possesso di artefatti quali token, credenziali o capability. In questo lavoro sosteniamo che il possesso non sia sufficiente per le catene di esecuzione discrete, tanto quando attraversano più servizi quanto quando si articolano in passi distinti all'interno della stessa macchina: esso, infatti, non garantisce che si conservi la relazione causale fra l'origine di una richiesta e l'autorità esercitata nei passi successivi. Introduciamo la Prova di Continuità, una disciplina minimale per la propagazione dell'autorità nel modello Provenance Identity Continuity (PIC). In tale disciplina, ogni passo di esecuzione deve essere legato causalmente al precedente e può propagare soltanto un sottoinsieme non espansivo dell'autorità ricevuta dall'origine. Il modello introduce inoltre la Prova di Relazione, una primitiva causale relativa a un singolo passaggio, la cui composizione transitiva costituisce la Prova di Continuità; entrambe integrano la Prova di Possesso, senza sostituirla. Nel modello proposto, la condizione del delegato confuso non può verificarsi come comportamento valido: ogni privilegio esercitato in un passo successivo deve essere già presente nel contesto di autorità dell'origine. Il risultato è direttamente pertinente ai sistemi distribuiti e agli agenti di IA, nei quali gli esecutori invocano strumenti e servizi a valle pur disponendo di più fonti di autorità, cosicché il medesimo disallineamento fra autorità e causalità si ripresenta oltre i confini dei servizi. Con la Prova di Continuità queste fonti possono essere trasportate insieme, ma non vengono mai fuse in un'unica autorità combinata, poiché ciascun passo è autorizzato esclusivamente rispetto al contesto di autorità della linea causale che lo ha determinato.

Il lavoro riguarda la propagazione dell'autorizzazione, non l'autenticazione: meccanismi di identità e autenticazione quali OIDC, credenziali

verificabili, wallet e identità dei carichi di lavoro restano strumenti complementari per stabilire l'origine; la Prova di Continuità disciplina invece il modo in cui l'autorità si propaga una volta stabilita tale origine.

1 Modello

Formalizziamo il modello PIC (Provenance Identity Continuity) come segue.

Siano P un insieme finito di soggetti, O un insieme di operazioni e R un insieme di risorse. Definiamo privilegio ogni coppia $(o, r) \in O \times R$.

A ogni soggetto p è associato un insieme di privilegi:

$$Priv(p) \subseteq O \times R.$$

Qui un soggetto è un'entità astratta dotata di permessi: può trattarsi di un'identità umana stabilita mediante un fornitore di identità, un wallet, un flusso sul modello di OIDC/OIDC4VC [13, 14] o un identificatore decentralizzato [15]; di un'identità di carico di lavoro o di macchina, come quelle impiegate negli ambienti di tipo WIMSE/SPIFFE [11, 12]; di un ruolo; di un account di servizio; oppure di un'altra entità autenticata e dotata di permessi. Ai fini del modello contano due proprietà: che all'entità sia associato un insieme di privilegi $Priv(p)$ e che essa possa esprimere un intento entro tali privilegi. Quando un'entità p esprime un simile intento, seleziona un sottoinsieme di $Priv(p)$; questo sottoinsieme diventa il contesto di autorità iniziale $C_0 \subseteq Priv(p)$ di una nuova linea causale di esecuzione. I privilegi in $Priv(p)$ possono essere espressi al livello delle operazioni che il soggetto è autorizzato a *causare*, non soltanto di quelle che compie direttamente; un passaggio a valle può realizzare tale autorità impiegando un vocabolario locale di operazioni differente, mediante la traduzione di policy $\mathcal{T}$ della Sezione 5.

Supponiamo, per esempio, che Bob disponga dei privilegi (read, Book) e (write, Book). Quando esprime un intento, Bob ne seleziona un sottoinsieme — poniamo il solo (write, Book) — che diventa il contesto di autorità iniziale C_0 di una nuova linea causale. Due esecuzioni possono coinvolgere entrambe Bob e il medesimo privilegio selezionato senza per questo costituire la stessa occorrenza: ciascuna trasporta l'autorità propagata lungo la propria linea causale. Una può essere la continuazione valida di un intento espresso da Bob entro una data linea; l'altra può essere un'esecuzione indipendente che esercita lo stesso privilegio apparente al di fuori della linea che lo ha conferito. Le coordinate dell'identità e del privilegio possono coincidere; è la coordinata della linea causale a distinguere le due esecuzioni.

Una richiesta è un messaggio *req*, eventualmente dotato di un carico utile, trasmesso fra esecutori, cioè le entità che compiono i passi di esecuzione.

L'esecuzione è modellata come una catena causale di passi:

$$\pi = \langle (s_0, C_0), (s_1, C_1), \dots, (s_n, C_n) \rangle$$

ove s_0 è il passo iniziale e $C_i \subseteq O \times R$ è il contesto di autorità trasportato al passo s_i . Indichiamo con s_i il passo di esecuzione formale al passaggio i ; sul piano operativo, un passaggio consiste nell'esecuzione di uno di tali passi da parte di un esecutore che riceve, elabora ed eventualmente propaga una richiesta. Ogni passaggio può essere compiuto da un servizio, un carico di lavoro, una funzione, uno strumento o un agente; il modello richiede la continuità dell'autorità, non la propagazione dell'identità.

Definizione 1 (Modello PIC). *Un sistema garantisce la Provenance Identity Continuity se, per ogni catena di esecuzione π , ciascuna transizione conserva la continuità causale e la restrizione monotona dell'autorità:*

$$C_{i+1} \subseteq C_i$$

per ogni i , e se ogni privilegio esercitato al passaggio n è collegato causalmente all'origine mediante una catena verificabile.

Il legame causale relativo a un singolo passaggio è attestato a ogni transizione è una *Prova di Relazione (PoR)*: un'evidenza che certifica come un passo sia la continuazione valida del suo predecessore immediato entro la stessa linea causale. Il modello PIC combina dunque due condizioni — il legame causale, attestato passaggio per passaggio da *PoR*, e la restrizione monotona $C_{i+1} \subseteq C_i$ —, qui presentate informalmente e definite in modo formale nella Sezione 5, dove la continuità è costruita come composizione transitiva di tali relazioni. Il teorema seguente enuncia la conseguenza della seconda condizione sull'autorità.

Teorema 1 (Sicurezza PIC). *Se vale il modello PIC, nessun passo di esecuzione può esercitare un privilegio assente all'origine.*

Dimostrazione. Per costruzione, $C_{i+1} \subseteq C_i$ per ogni i , dunque $C_n \subseteq C_0$. Ne segue che ogni $(o, r) \in C_n$ appartiene anche a C_0 . Non è quindi possibile acquisire alcun privilegio ulteriore rispetto al contesto di autorità iniziale.

2 Modello di minaccia e ambito

Consideriamo catene di esecuzione nelle quali una richiesta iniziale determina uno o più passi di esecuzione successivi. Gli esecutori possono contenere errori, trovarsi in uno stato di confusione oppure subire un'influenza avversaria attraverso i propri input o il proprio contesto di esecuzione. Un esecutore può disporre di più fonti di autorità — privilegi ambientali, token delegati, credenziali al portatore o capability — e può tentare di usarne una estranea alla richiesta che sta elaborando.

L'avversario può influenzare le richieste, i carichi utili e i parametri trasmessi lungo la catena, inducendo un esecutore a scegliere la fonte di autorità errata. Non può però falsificare l'evidenza di relazione richiesta da *PoR*, né indurre un verificatore di continuità valido ad accettare una transizione che viola la monotonia. Così come i modelli basati su capability e token assumono che le proprie credenziali non siano falsificabili, assumiamo non falsificabili l'evidenza di relazione e il meccanismo di transizione dell'autorità; la compromissione del validatore, della radice crittografica di fiducia o dell'inizializzazione dell'autorità esula dall'ambito del lavoro.

3 Delegato confuso

Definizione 2 (Delegato confuso). *Si verifica un caso di delegato confuso quando esistono due soggetti U (utente) e D (delegato) e un privilegio $(o, r) \in O \times R$ tali che:*

1. $(o, r) \notin Priv(U)$,
2. $(o, r) \in Priv(D)$,
3. U invia una richiesta req a D ,
4. in conseguenza di req , D esegue (o, r) .

La definizione non dipende dai dettagli implementativi, ma soltanto dal disallineamento fra autorità e causalità.

Esempio classico. Un compilatore FORTRAN FORT, installato nella directory privilegiata SYSX, dispone dell'autorità ambientale necessaria a scrivere le statistiche in (SYSX)STAT. Un utente invoca il compilatore e indica (SYSX)BILL come nome del file di output per il debug. Il compilatore apre in

scrittura il file di destinazione avvalendosi della propria autorità su **SYSX**, sovrascrivendo così **(SYSX)BILL**. L'utente non ha mai posseduto tale privilegio, ma il delegato lo ha esercitato in conseguenza della richiesta dell'utente [1].

4 Prova di Possesso (PoP)

Un token t rappresenta un insieme di privilegi:

$$Priv(t) \subseteq O \times R.$$

Semantica di PoP. Il possesso implica la possibilità d'uso: se un soggetto possiede t , può esercitare ogni $(o, r) \in Priv(t)$.

I sistemi PoP vincolano l'autorità al possesso di un artefatto, ma non richiedono, di per sé, che l'autorità esercitata sia una continuazione causale del contesto di autorità della richiesta.

4.1 Token vincolati al mittente e al possessore

I token vincolati al mittente o al possessore — per esempio DPoP [9] e Token Binding [10] — dimostrano *chi* possa presentare o usare un artefatto, legandolo a una chiave o a un canale di trasporto. Non dimostrano però, da soli, che l'autorità esercitata tramite tale artefatto continui l'attuale linea causale di esecuzione. Limitare chi possa presentare un token pone un vincolo sul trasporto, non sulla causalità: in un dato passo, un possessore legittimo può comunque esercitare un'autorità in suo possesso ma esterna alla linea causale della richiesta che sta elaborando. Il vincolo risponde alla domanda sull'autenticità del possessore; resta invece aperta la questione causale, ossia se questo uso prosegua il contesto di autorità che lo ha determinato.

4.2 Commistione di autorità

Supponiamo che:

$$(write, r) \notin Priv(U) \tag{H1}$$

$$(write, r) \in Priv(D) \tag{H2}$$

Qui U è l'utente e D un delegato.

In un modello basato sul possesso, un delegato può disporre di più fonti di autorità: i propri privilegi ambientali, token delegati, credenziali al portatore o altri artefatti. Il modello autorizza un'operazione verificando se una delle fonti di autorità possedute concede il privilegio richiesto.

Definizione 3 (Funzione di selezione). *Sia $\{A_j\}$ la famiglia delle fonti di autorità detenute da un esecutore — privilegi ambientali, token delegati e artefatti vincolati al possessore o al mittente. Una funzione di selezione σ sceglie, per un'operazione richiesta (o, r) , una fonte A_j tale che $(o, r) \in A_j$, indipendentemente dalla linea causale ℓ della richiesta — ossia la linea causale della richiesta in corso di elaborazione, formalizzata nella Sezione 5.*

Teorema 2 (PoP ammette la commistione di autorità). *In un modello basato sul possesso, se un esecutore può selezionare l'autorità da un qualsiasi artefatto o privilegio ambientale di cui dispone, indipendentemente dal contesto di autorità della richiesta, allora il modello ammette la condizione del delegato confuso.*

Dimostrazione. Consideriamo una richiesta che ha origine in un contesto di autorità privo di (o, r) . Supponiamo che l'esecutore disponga di un altro contesto di autorità, artefatto o privilegio ambientale che invece contiene (o, r) .

In un modello PoP, l'autorizzazione dipende dal possesso di un artefatto o di un privilegio che concede (o, r) , non dal fatto che tale autorità continui il contesto di autorità della richiesta. Il modello consente dunque un'esecuzione nella quale la richiesta determina causalmente (o, r) benché (o, r) sia assente dal contesto di autorità della richiesta. Questa è precisamente la condizione del delegato confuso della Definizione 2.

Il ragionamento vale per ogni modello nel quale l'autorizzazione discende dal possesso indipendentemente dalla linea causale, compresi i JWT, le credenziali al portatore e le capability trattate come artefatti posseduti quando possono essere applicate al di fuori dell'esecuzione che le ha richieste. Il vincolo al possessore e quello al mittente limitano quale soggetto possa presentare A_j , non stabiliscono se la scelta operata da σ appartenga a ℓ ; pertanto non eliminano tale ammissibilità. La lacuna non risiede nel possesso in sé, ma nell'assenza di un vincolo temporale fra l'autorità impiegata e l'esecuzione che ne ha causato l'uso.

Si tratta di condizioni al livello dell'esecuzione: l'autorità appartenente a una linea causale viene introdotta in un'altra, dando luogo al delegato confuso. La Prova di Continuità elimina questa ammissibilità al livello del modello: in un'esecuzione PIC valida la condizione del delegato confuso non può verificarsi (Sezione 7), poiché l'autorità è definita esclusivamente come continuazione della linea causale che l'ha determinata.

5 Prova di Continuità (PoC / PIC)

L'esecuzione è una catena causale:

$$s_0 \rightarrow s_1 \rightarrow \cdots \rightarrow s_n.$$

Indichiamo con $C_i \subseteq O \times R$ il contesto di autorità trasportato dal passo di esecuzione s_i .

Definizione 4 (Prova di Relazione). *PoR(s_i, s_{i+1}) vale se e solo se s_{i+1} è una continuazione causale valida di s_i entro la medesima linea causale di esecuzione. PoR è una proprietà relazionale, riferita a un singolo passaggio: lega ciascun passo al suo predecessore immediato. Il collegamento retrospettivo con l'origine non è contenuto in una singola PoR, ma si ottiene componendo relazioni consecutive, come precisa la definizione di PoC riportata di seguito.*

Definizione 5 (Transizione valida). *Una transizione da s_i a s_{i+1} è valida se e solo se è al tempo stesso legata causalmente e non espansiva:*

$$Valid(s_i, C_i, s_{i+1}, C_{i+1}) \iff PoR(s_i, s_{i+1}) \wedge C_{i+1} \subseteq C_i.$$

La prima condizione dimostra la relazione causale; la seconda dimostra la restrizione monotona dell'autorità. Ogni passaggio trasferisce quindi soltanto:

$$C_{i+1} \subseteq C_i. \tag{C1}$$

Definizione 6 (Prova di Continuità). *Per una catena $\pi = \langle s_0, \dots, s_n \rangle$, PoC(π) vale se e solo se ogni transizione fra passi adiacenti è valida:*

$$PoC(\pi) \iff \bigwedge_{i=0}^{n-1} (PoR(s_i, s_{i+1}) \wedge C_{i+1} \subseteq C_i).$$

La continuità è dunque la composizione transitiva delle relazioni: PoR dimostra un singolo legame, PoC l'intera linea causale. La relazione è locale; la continuità è globale.

Propagazione monotona lungo una catena causale. Due elementi si propagano congiuntamente. In primo luogo, l'autorità si attenua in modo monotono: scriviamo $C_{i+1} \preceq C_i$ per indicare l'*attenuazione* dell'autorità — ogni autorità presente a un passaggio è una restrizione di quella presente al passaggio precedente, mai un'espansione. Quando il contesto è un insieme di privilegi atomici, la relazione si specializza in $C_{i+1} \subseteq C_i$, che è la forma

usata nei nostri esempi. Sebbene il nucleo del modello rappresenti i contesti di autorità come insiemi di privilegi atomici, usiamo $\preceq$ per riferirci all'ordine generale di attenuazione; nel caso insiemistico adottato in tutti gli esempi formali, tale ordine coincide con $\subseteq$. In secondo luogo, PoR lega i passaggi in un'unica linea causale: la continuità è proprietà non di un passaggio isolato, bensì della *catena*. Formalmente, la linea è una catena causale logica; un'implementazione non deve necessariamente trasportarla per intero: può rappresentarla mediante commitment crittografici, riferimenti, hash, firme o prove sintetiche che consentano a ogni passaggio di dimostrare la propria posizione nella linea senza trasportarne l'intera storia.

Inserimento dell'esecuzione nel modello di autorità. La Prova di Continuità è definita proprio per questo scenario: non è necessario che il passo successore sia noto, scelto, istanziato o predisposto quando il predecessore termina il proprio passo. L'esecuzione è una sequenza temporale di passi causali, non di posizioni fissate dalla topologia: l'esecutore a monte agisce a un istante t , mentre quello a valle può venire a esistenza soltanto a un istante successivo $t' > t$. Non è quindi sempre possibile vincolare in anticipo l'autorità a uno specifico possessore, a una chiave o al canale di un successore ancora ignoto.

Ciò che deve conservarsi oltre tale discontinuità non è l'identità del successore, né una policy a esso associata, bensì il fatto che la sua autorità continui il contesto che lo ha determinato. Una credenziale, un'attestazione dell'identità del carico di lavoro, una credenziale di account di servizio o un token vincolato possono autenticare il possessore oppure legare un artefatto a una chiave, a un carico di lavoro o a un canale. Tuttavia, se la decisione di autorizzazione non considera anche evidenze sensibili alla linea causale, i medesimi attributi del possessore possono presentarsi in esecuzioni distinte: essi stabiliscono chi agisce, non quale catena causale abbia prodotto la richiesta. Derivare nuovamente l'autorità in $i + 1$ da un'asserzione nuova sul successore, anziché dalla sua relazione con il passo che lo ha determinato, reintroduce la lacuna di individuazione della Sezione 5.3 — la condizione strutturale alla base del delegato confuso (Definizione 2). La continuità evita il problema perché non può essere autoasserita: essendo definita soltanto in relazione al predecessore, può essere *dimostrata* come continuazione, ma non asserita isolatamente in $i + 1$.

Un predecessore può invece emettere una continuazione che sarà poi consumata da un successore:

$$(s_i, C_i) \xrightarrow{\text{emit}} \text{unknown next hop} \xrightarrow{PoR, C_{i+1} \subseteq C_i} (s_{i+1}, C_{i+1}).$$

Oltre tale discontinuità persiste una continuità valida, non il possesso da parte di un titolare preesistente: persistono una relazione causale con il predecessore e un contesto di autorità non espansivo. La Prova di Relazione attesta tale proprietà dell'esecuzione, anziché del possessore; composta transitivamente nel rispetto dell'attenuazione monotona ($C_{i+1} \preceq C_i$), essa costituisce la Prova di Continuità, cioè l'invariante di propagazione del modello PIC.

Spazi di operazioni eterogenei. La notazione presuppone un universo comune dei privilegi $O \times R$. Nei sistemi eterogenei, passaggi diversi impiegano vocabolari differenti; sia $O_i \times R_i$ l'universo al passaggio i . Una transizione è allora regolata da una traduzione di policy

$$\mathcal{T}_{i \rightarrow i+1} : \mathcal{P}(O_i \times R_i) \rightarrow \mathcal{P}(O_{i+1} \times R_{i+1}),$$

e la monotonia diventa $C_{i+1} \subseteq \mathcal{T}_{i \rightarrow i+1}(C_i)$: l'autorità del successore non supera quella consentita dal predecessore secondo la traduzione locale. Consideriamo $\mathcal{T}$ un dato di policy fissato; la sintesi e la verifica di tali traduzioni esulano dall'ambito del lavoro. L'eterogeneità è qui semantica, non meramente amministrativa: passaggi diversi possono descrivere l'autorità mediante vocabolari di operazioni e risorse differenti persino entro un unico perimetro di fiducia. Nei sistemi eterogenei, la sicurezza è pertanto relativa alla correttezza della traduzione di policy scelta $\mathcal{T}_{i \rightarrow i+1}$; una traduzione eccessivamente permissiva costituisce un errore di policy, non una violazione dell'invariante di continuità in sé.

Lemma 1 (La continuità implica il vincolo all'origine). *Se vale $PoC(\pi)$, allora s_n è collegato causalmente a s_0 mediante una catena relazionale verificabile e $C_n \subseteq C_0$.*

Dimostrazione. Ogni $PoR(s_i, s_{i+1})$ collega due passi consecutivi; la loro composizione collega s_n a s_0 . La non espansività a ogni passaggio dà $C_n \subseteq \dots \subseteq C_0$.

Precisiamo esplicitamente l'ambito: il lavoro modella l'invariante di continuità; la costruzione concreta (per esempio crittografica) di PoR esula da tale ambito ed è realizzata da un'architettura di applicazione complementare. Così come i modelli basati su capability e token assumono non falsificabili le proprie credenziali, assumiamo non falsificabile la linea causale.

5.1 Regola di autorizzazione

La regola seguente viene valutata soltanto per una catena di continuità valida, ossia una catena nella quale ogni transizione fra passi adiacenti soddisfa sia $PoR(s_i, s_{i+1})$ sia $C_{i+1} \subseteq C_i$.

Nell'universo comune dei privilegi $O \times R$, (o, r) è autorizzato al passaggio finale se e solo se:

$$(o, r) \in \bigcap_{i=0}^n C_i.$$

Poiché la catena è monotona decrescente:

$$C_n \subseteq C_{n-1} \subseteq \dots \subseteq C_0,$$

si ha:

$$\bigcap_{i=0}^n C_i = C_n.$$

Nessun passaggio può quindi acquisire un'autorità che non fosse già presente all'origine, e ogni autorità abbandonata a un certo passaggio j è perduta in modo irreversibile: per monotonia, non può ricomparire in alcun passaggio successivo.

5.2 Il cambiamento ontologico: possesso e continuità

I modelli basati sul possesso e sulle capability [2, 3] rispondono a una domanda *spaziale*: un attore detiene autorità su un oggetto? L'autorità è un punto di $O \times R$. Il modello è atemporale: non possiede alcun asse sul quale registrare l'esecuzione che ha condotto a un'azione.

La Prova di Relazione fornisce l'asse mancante, legando un passo di esecuzione al suo predecessore causale. Su di essa si costruisce la Prova di Continuità — la continuità è la composizione transitiva delle relazioni —, che combina due condizioni: la restrizione monotona dei privilegi ($C_{i+1} \subseteq C_i$) e, mediante PoR , la continuità causale dell'esecuzione che li trasporta.

L'autorità non appartiene dunque più al solo $O \times R$, bensì a $O \times R \times \mathcal{L}$, dove $\mathcal{L}$ è la linea causale dell'esecuzione. In concreto, $\mathcal{L}$ è l'insieme delle linee causali lineari finite: una linea è una sequenza $\ell = \langle s_0 \prec s_1 \prec \dots \prec s_n \rangle$, nella quale $\prec$ denota la precedenza causale, attestata fra passi adiacenti da PoR ; ogni evento è così un'occorrenza di un privilegio entro una particolare linea causale. Due azioni identiche quanto al possesso — stesso soggetto, stesso token, stesso privilegio — possono differire quanto alla continuità, perché discendono da cause diverse. Il medesimo (o, r) , esercitato come

continuazione valida di una richiesta autorizzante, oppure da un esecutore che attinge a un'autorità estranea di cui casualmente dispone, è indistinguibile dal punto di vista del possesso; si tratta tuttavia di due esecuzioni distinte: la prima continua una linea causale autorizzante, la seconda non ne continua alcuna. Il possesso è la proiezione su $O \times R$ che dimentica l'asse della linea causale. Esso è la componente spaziale dell'autorità, mentre la linea causale ne è la componente temporale; un modello basato sul solo possesso è pertanto una proiezione dell'autorità che dimentica il tempo. Ciò non rende errati i modelli basati sul possesso, ma li rende incompleti nel caso multipassaggio, nel quale la continuazione dell'autorità diventa una proprietà temporale. L'evento rilevante non è dunque il solo (p, o, r) , ossia identità più privilegio, bensì un'occorrenza di un privilegio entro una linea causale. Quando il medesimo soggetto e il medesimo privilegio ricorrono in due linee diverse, la coordinata dell'identità non li distingue; quella della linea causale sì.

Nel linguaggio delle proiezioni, questa è la *lettura temporale* del delegato confuso della Definizione 2: un'azione valida nello spazio ma non nel tempo — il delegato dispone di (o, r) , ma $(o, r) \notin C_0$ per la richiesta che l'ha causata —, cioè una collisione nella proiezione che dimentica $\mathcal{L}$: due esecuzioni distinte che il possesso non è in grado di distinguere. La Sezione 6 fornisce la caratterizzazione operativa (il *tramite aperto*), affinando la Definizione 2 mediante la designazione.

Relazione con le capability. Le capability risolvono la questione spaziale e vanno oltre: fondendo designazione e autorità nell'atto dell'invocazione, una capability lega l'autorità a un singolo passo causale — nei nostri termini, una *continuità a passaggio singolo*, cioè una Prova di Relazione —, cosicché il delegato confuso classico è escluso, non semplicemente mitigato. Ciò che manca a una capability trattata unicamente come artefatto posseduto è l'estensione di questo vincolo lungo una catena: se applicata al di fuori della linea causale che l'ha richiesta, essa ricade nel possesso invariante rispetto alla linea.

Prova di Possesso e Prova di Continuità sono due primitive di propagazione dell'autorità. L'invocazione di una capability è un'istanza a singolo passaggio della primitiva di continuità, mentre una capability detenuta e presentata come artefatto si fonda sul possesso; un sistema di capability che trasportasse la continuità lungo l'intera catena sarebbe, in questi termini, conforme a PIC. PIC è dunque un raffinamento temporale della propagazione basata sulle capability, non un suo sostituto: introduce una dimensione che il solo possesso non può codificare. Entrambi eliminano per costruzione il

delegato confuso, anziché affidarsi alla vigilanza in fase di esecuzione, ma operano su estensioni diverse. Nel modello a capability di oggetto, la fusione di designazione e autorità al momento dell'invocazione elimina strutturalmente il classico disallineamento a singolo passaggio: un delegato non può applicare autorità a una destinazione per la quale non l'ha ricevuta. Il disallineamento riappare soltanto quando si esce dal modello a capability — in presenza di autorità ambientale, oppure quando una capability è usata come artefatto posseduto al di fuori dell'occorrenza che l'ha trasportata: il caso, analizzato sopra, della capability come possesso, nel quale l'autorità torna a dipendere dal possesso. PIC estende la stessa garanzia per costruzione a una linea causale multipassaggio: il delegato confuso non è uno stato rappresentabile in alcun passaggio di un'esecuzione PIC valida, perché un privilegio assente dal contesto di autorità iniziale non costituisce affatto uno stato successivo valido.

5.3 Proiezione: il possesso invariante rispetto alla linea non può distinguere le occorrenze

Precisiamo la proiezione. Un *evento* è un'occorrenza che esercita un privilegio entro una linea causale,

$$e = (o, r, \ell) \in E := O \times R \times \mathcal{L},$$

e $p : E \rightarrow O \times R$, $p(o, r, \ell) = (o, r)$, è la proiezione che dimentica la linea causale.

Definizione 7 (Policy basata sul possesso). *Una policy di autorizzazione $A : E \rightarrow \{0, 1\}$ è basata sul possesso se e solo se è invariante rispetto alla linea causale: essa si fattorizza attraverso p , cioè esiste $\bar{A} : O \times R \rightarrow \{0, 1\}$ tale che $A = \bar{A} \circ p$. Equivalentemente, $A(o, r, \ell) = A(o, r, \ell')$ per ogni $\ell, \ell' \in \mathcal{L}$: il verdetto non dipende dalla linea che ha prodotto l'evento.*

Teorema 3 (Le policy invarianti rispetto alla linea non distinguono le occorrenze). *Siano $e = (o, r, \ell)$ ed $e' = (o, r, \ell')$ due eventi con il medesimo privilegio ma linee causali distinte, $p(e) = p(e')$ e $\ell \neq \ell'$. Allora ogni policy basata sul possesso soddisfa $A(e) = A(e')$.*

Dimostrazione. $A = \bar{A} \circ p$ e $p(e) = p(e')$, pertanto $A(e) = \bar{A}(p(e)) = \bar{A}(p(e')) = A(e')$.

Il delegato confuso è un caso particolare di coppia vietata: due eventi che condividono (o, r) ma differiscono per linea causale, uno dei quali deve essere

autorizzato e l'altro negato. Una linea ha origine in un contesto che contiene (o, r) ; l'altra esercita (o, r) senza aver mai continuato un contesto che lo contenesse. Per il teorema, *nessuna* policy basata sul possesso e invariante rispetto alla linea può distinguerli.

Teorema 4 (Ogni soluzione reintroduce la continuità). *Una policy A autorizza e e nega e' , con $p(e) = p(e')$ e $\ell \neq \ell'$, soltanto se A non è invariante rispetto alla linea; in altri termini, A deve leggere una funzione $g : \mathcal{L} \rightarrow X$ della linea causale tale che $g(\ell) \neq g(\ell')$.*

Dimostrazione. Se A fosse invariante rispetto alla linea, si fattorizzerebbe attraverso p e, per il teorema precedente, darebbe $A(e) = A(e')$, in contraddizione con $A(e) \neq A(e')$. Dunque A dipende dalla coordinata della linea causale: esiste una funzione $g(\ell)$ che distingue e da e' .

Una tale g è esattamente un meccanismo di continuità, ossia un'estrazione dalla linea causale che funge da prova di relazione. Le mitigazioni impiegate nei sistemi basati sul possesso (nonce, chiavi di idempotenza, marcature temporali, vincoli al destinatario e al contesto) sono istanze concrete di g : risolvono il delegato confuso soltanto reintroducendo nella decisione l'informazione sulla linea causale, cioè la continuità. È irrilevante che la linea sia trasportata insieme alla richiesta o incorporata nell'artefatto: un token che codifica un nonce o un contesto causale rende A dipendente da ℓ e dunque, per definizione, sensibile alla continuità, non basato sul possesso. La distinzione non riguarda il luogo in cui risiede l'informazione, bensì il fatto che la decisione la legga o meno. Il limite non è un difetto ingegneristico dei sistemi basati sul possesso, ma un confine modellistico dell'autorizzazione invariante rispetto alla linea: per il primo teorema, essa è strutturalmente incapace di distinguere le occorrenze, e ogni rimedio efficace è sensibile alla linea — dunque alla continuità — anziché invariante rispetto a essa.

Perché il modello del possesso è rappresentativo. Nell'astrazione qui considerata, un token al portatore usato senza un vincolo esplicito al contesto o alla linea causale è invariante rispetto a quest'ultima: la decisione legge l'artefatto e il privilegio richiesto, non la linea causale del suo uso. Un JWT, per esempio, asserisce che “il possessore può leggere”, non quale catena causale abbia condotto a tale lettura. Questi sistemi spesso funzionano non perché codifichino esplicitamente la continuità, ma perché un perimetro ne fornisce implicitamente una parte: TLS mutuo, confini di rete e delegazione desunta dal trasporto suppliscono spesso alla linea causale mancante. Quando il perimetro viene meno — nelle installazioni

zero-trust, nelle catene fra più organizzazioni sulla rete pubblica e negli agenti autonomi di IA che invocano strumenti e servizi —, l'invarianza rispetto alla linea si manifesta come vulnerabilità al secondo passaggio. La definizione di policy basata sul possesso data sopra descrive dunque sistemi effettivamente impiegati, non una loro caricatura. Un perimetro di fiducia può celare la lacuna temporale facendo apparire autorità e causalità come co-localizzate, ma il modello non presuppone l'attraversamento di un simile perimetro: il disallineamento può sorgere ogni volta che l'autorità viene scelta indipendentemente dall'esecuzione che ne ha causato l'uso.

5.4 Un teorema di compromesso per la propagazione dell'autorità

La proiezione della Sezione 5.3 costituisce il fondamento tecnico di un compromesso fra tre proprietà che si può richiedere a un sistema di propagazione dell'autorità di soddisfare simultaneamente.

Definizione 8 (Autorizzazione invariante rispetto alla linea). *L'autorizzazione di un evento $e = (o, r, \ell)$ dipende soltanto dall'autorità posseduta per (o, r) , non dalla linea causale ℓ che ha determinato l'evento.*

Definizione 9 (Delegazione che consente la commistione di autorità). *Un esecutore può disporre sia di un contesto di autorità C_0 derivato dalla richiesta, sia di una fonte di autorità indipendente C_D , con un (o, r) tale che $(o, r) \notin C_0$ e $(o, r) \in C_D$.*

Definizione 10 (Sicurezza rispetto al delegato confuso). *Nessuna richiesta il cui contesto iniziale C_0 non contenga (o, r) può indurre un esecutore successivo a esercitare (o, r) mediante un'autorità non derivata dalla richiesta stessa.*

Teorema 5 (Compromesso fra possesso, delegazione e sicurezza). *Nessun sistema di propagazione dell'autorità può soddisfare simultaneamente l'autorizzazione invariante rispetto alla linea, la delegazione che consente la commistione di autorità e la sicurezza rispetto al delegato confuso.*

Dimostrazione. Supponiamo che valgano tutte e tre. Per la delegazione che consente la commistione di autorità, esistono un esecutore D , un contesto C_0 derivato dalla richiesta, una fonte indipendente C_D e un (o, r) tali che $(o, r) \notin C_0$ e $(o, r) \in C_D$. Sia data una richiesta che induce D a elaborare una destinazione per la quale occorre (o, r) . L'autorizzazione invariante rispetto alla linea rileva che D possiede C_D , che concede (o, r) ; tuttavia,

per il teorema di proiezione della Sezione 5.3, eventi che condividono (o, r) ma differiscono per linea causale ricevono lo stesso giudizio. La policy non può quindi dipendere dal fatto che C_D appartenga o meno alla linea della richiesta in esame. Il sistema ammette pertanto un'esecuzione nella quale la richiesta induce D a esercitare (o, r) nonostante $(o, r) \notin C_0$, violando la sicurezza rispetto al delegato confuso: una contraddizione.

Il teorema non costituisce una critica al possesso: mostra che il possesso invariante rispetto alla linea è insufficiente per la propagazione multipassaggio dell'autorità proprio quando si richiedono sia la commistione di autorità sia la sicurezza rispetto al delegato confuso. Poiché le tre proprietà non possono valere insieme, un sistema deve rinunciare ad almeno una di esse. Per i sistemi che richiedono sia una commistione di autorità concretamente utilizzabile, sia la sicurezza rispetto al delegato confuso, la scelta praticabile è abbandonare l'invarianza rispetto alla linea, rendendo l'autorizzazione sensibile a essa. Vietare la delegazione che consente la commistione impedirebbe invece agli esecutori di detenere autorità propria insieme a quella derivata dalla richiesta: una soluzione impraticabile nelle installazioni reali, dove un servizio conserva legittimamente i propri privilegi. Accettare le esecuzioni con delegato confuso significherebbe rinunciare alla proprietà di sicurezza in esame. La Prova di Continuità sceglie la prima opzione: non elimina il possesso, ma ripristina la coordinata temporale mancante, permettendo all'autorizzazione di verificare se ogni uso di autorità costituisca una continuazione valida del contesto che lo ha determinato. Il contributo non consiste nel rifiutare il possesso, bensì nel mostrare che esso è la proiezione spaziale di una relazione di autorità più ricca; la coordinata mancante è la linea causale temporale.

Corollario 1 (La continuità ripristina la sicurezza rispetto al delegato confuso). *Abbandonando l'invarianza rispetto alla linea si recupera la sicurezza: con la propagazione non espansiva $C_{i+1} \subseteq C_i$, dunque $C_k \subseteq C_0$, e un $(o, r) \notin C_0$ non può comparire in alcun passaggio successivo. È l'impossibilità stabilita nella Sezione 7.*

6 Proprietà di sicurezza

Definizione 11 (Autorità limitata dall'origine). *Un modello garantisce un'autorità limitata dall'origine se ogni privilegio eseguibile al passaggio n era stato concesso in origine al passaggio 0.*

Affiniamo la Definizione 2 per distinguere la delegazione legittima dal delegato confuso. La variabile discriminante non è la fonte dell'autorità, ma

Proprietà abbandonata	Conseguenza	Esito
Delegazione con commistione di autorità	gli esecutori non possono detenere autorità indipendente dalla richiesta	respinta (impraticabile)
Sicurezza rispetto al delegato confuso	diventano ammissibili esecuzioni con delegato confuso	respinta (non sicura)
Invarianza rispetto alla linea	l'autorizzazione diventa sensibile alla linea	adottata (Prova di Continuità)

Tabella 1: Le tre proprietà non possono valere contemporaneamente, perciò un sistema deve rinunciare ad almeno una di esse. Per i sistemi che richiedono una commistione di autorità concretamente utilizzabile e la sicurezza rispetto al delegato confuso, la scelta praticabile è rinunciare all'invarianza rispetto alla linea: l'opzione adottata dalla Prova di Continuità.

il fatto che la destinazione dell'azione sia designata dalla richiesta e che il richiedente fosse autorizzato a determinarla.

Definizione 12 (Autorità di causare). *Il contesto iniziale C_0 registra l'autorità che il richiedente è autorizzato a causare, non soltanto a esercitare direttamente. Un'azione successiva appartiene alla linea causale del richiedente se e solo se realizza un'autorità in C_0 ; altrimenti è un'azione propria del delegato, radicata in un'origine distinta (il delegato stesso) e non vincolata da C_0 .*

Definizione 13 (Tramite aperto). *Un delegato è un tramite aperto quando, in conseguenza di una richiesta, applica autorità a una destinazione designata dalla richiesta stessa per un'azione che il richiedente non era autorizzato a causare: $(o, r) \notin C_0$, ma (o, r) viene comunque eseguito a causa della richiesta. Questo è esattamente il delegato confuso.*

Le definizioni separano tre casi. (i) Le attività interne del delegato — il compilatore che scrive il proprio file di statistiche — hanno origine nel delegato stesso e non sono vincolate da C_0 . (ii) La delegazione legittima — un gateway di pagamento che trasferisce fondi perché l'utente era autorizzato a causare il pagamento — realizza un'autorità in C_0 ed è una continuazione valida, sebbene l'utente non detenga direttamente l'autorità bancaria. (iii) Il delegato confuso — la scrittura di un file designato dall'utente, che questi non poteva causare — è un tramite aperto: l'unico caso vietato dal modello. La domanda rilevante non è se l'origine potesse compiere direttamente

l'operazione a valle, ma se fosse autorizzata a *causarla* entro questa linea. Quando servizi successivi usano spazi di operazioni differenti, interviene la traduzione di policy $\mathcal{T}$ introdotta sopra, la cui sintesi e verifica sono rinviate a lavori futuri.

Esempio. Se $C_0 = \{(\text{convert}, r)\}$, allora (write, r) non può mai essere autorizzato entro la medesima catena di esecuzione: $(\text{write}, r) \notin C_0$ e $C_{i+1} \subseteq C_i$ a ogni transizione, pertanto $(\text{write}, r) \notin C_n$.

7 Impossibilità del delegato confuso

La condizione del delegato confuso richiede un disallineamento di autorità: un privilegio assente dal contesto di autorità della richiesta viene nondimeno esercitato in conseguenza della richiesta stessa.

È il secondo risultato d'impossibilità del lavoro. La Sezione 5.3 mostra che il possesso *non può* distinguere le occorrenze; qui mostriamo il risultato duale: in presenza della continuità, il delegato confuso *non è uno stato rappresentabile dal modello*. La condizione non è soddisfacibile in alcuna esecuzione PIC valida: si tratta di un'impossibilità per costruzione, al livello dei comportamenti validi del modello.

Teorema 6 (Il delegato confuso è impossibile nel modello PIC). *In ogni esecuzione PIC valida — cioè un'esecuzione le cui transizioni adiacenti soddisfano PoR e la monotonia $C_{i+1} \subseteq C_i$, cosicché l'autorità è limitata dall'origine ($C_n \subseteq C_0$) — le condizioni del delegato confuso non possono essere soddisfatte congiuntamente come comportamento valido del modello.*

Dimostrazione. Per PoR, i passaggi formano un'unica linea causale radicata in s_0 e, per monotonia, $C_k \subseteq C_0$ per ogni k ; dunque $(o, r) \notin C_0$ implica $(o, r) \notin C_k$. L'autorità detenuta dal delegato al di fuori di questa linea ha un'origine distinta (Sezione 6) e non viene esercitata come parte della richiesta; entro la linea della richiesta, il disallineamento di privilegi necessario al delegato confuso è pertanto insoddisfacibile.

Interpretazione. In PIC l'autorità non è qualcosa che un soggetto semplicemente *possiede*; è una proprietà di continuità della catena di esecuzione. Poiché nessun passaggio può introdurre o reinterpretare un'autorità assente all'origine, un'esecuzione con delegato confuso può ancora essere tentata sul piano fisico, ma non può essere accettata come comportamento valido nel modello PIC.

8 Discussione

I sistemi PoP fanno intenzionalmente discendere l'autorizzazione dal controllo di un artefatto, proprietà spesso utile e necessaria per le decisioni locali di autorizzazione. In un'esecuzione multipassaggio, tuttavia, il solo controllo dell'artefatto non stabilisce se un uso successivo sia una continuazione valida della linea di autorità che lo ha determinato. I sistemi PIC/PoC propagano invece soltanto sottoinsiemi non espansivi dell'autorità lungo una catena di esecuzione verificata causalmente.

La Prova di Possesso dimostra il controllo dell'artefatto. La Prova di Relazione dimostra la relazione fra esecuzioni. La Prova di Continuità dimostra la continuazione dell'autorità. La Prova di Possesso stabilisce il controllo su un artefatto di autorità in un punto dell'esecuzione; da sola, non stabilisce che un uso successivo costituisca una continuazione valida della linea di autorità che lo ha determinato: è questa la condizione temporale aggiunta dalla Prova di Continuità. La Prova di Possesso può fungere da meccanismo per costruire una Prova di Relazione — il possesso di un artefatto di continuazione emesso dal predecessore può attestare un singolo passaggio —, ma non è l'unico meccanismo possibile.

L'autorità è continuità. Nel modello PIC, l'autorità non è una proprietà statica che un'entità si limita a *possedere*, bensì una proprietà di continuità dell'esecuzione: un privilegio è autorizzato in un passo se e solo se è una continuazione non espansiva del contesto che lo ha determinato. In questo senso l'autorità è continua: esiste soltanto come prosecuzione ininterrotta di un'origine, non come possesso separato dalla propria linea causale.

Spazio delle soluzioni progettuali. Il vincolo temporale può essere realizzato in due modi: trasportato *all'interno* dello stesso artefatto di autorità, come nelle capability attenuate e vincolate al contesto, oppure mantenuto *accanto* a esso sotto forma di prova esplicita di continuità. Entrambe le soluzioni possono essere sensibili alla continuità: per il Teorema 4, ciò che conta non è dove risieda il vincolo, ma se la decisione di autorizzazione lo legga. Un artefatto il cui vincolo venga ignorato — perché presentato al di fuori dell'occorrenza che lo ha trasportato — ricade invece nel possesso invariante rispetto alla linea. PIC adotta la seconda impostazione, trattando la continuità come proprietà di prima classe della catena di esecuzione; i due approcci sono complementari, non alternativi.

L'invariante di sicurezza risultante è semplice: nessun servizio a valle può compiere, come parte di una catena di esecuzione valida, un'operazione

non autorizzata all'origine. Si tratta di un affinamento del modello, non di una critica ai meccanismi esistenti: possesso, capability, token al portatore e vincoli di prova del possesso restano utili strumenti locali di autorizzazione, mentre PoC rende esplicita l'ulteriore condizione temporale necessaria quando l'autorità viene propagata lungo una linea causale di esecuzione.

L'autorità come transazione distribuita. Una catena multipassaggio di autorità è una transazione distribuita sull'autorità, anziché sui dati. Un artefatto posseduto consente una decisione locale di autorizzazione nel punto d'uso — spesso necessaria per l'installabilità, le prestazioni e il basso accoppiamento —, ma revoca, stato della delegazione, variazioni dell'ambito e linea causale sono fatti globali riguardanti l'autorità. Autorizzazione locale e coerenza globale dell'autorità divergono quando tali fatti non sono rappresentati nel punto d'uso. PoC affronta il problema trasportando esplicitamente la linea causale e imponendo una propagazione non espansiva dell'autorità lungo la catena.

La continuità non coincide quindi con la propagazione dell'identità. Fornitori di identità, wallet, credenziali verificabili [16], flussi sul modello di OIDC/OIDC4VC [13, 14] e identità dei carichi di lavoro sul modello di WIMSE/SPIFFE [11, 12] restano importanti per autenticazione, presentazione delle credenziali, audit, attribuzione di responsabilità e inizializzazione dell'origine; tuttavia, la ripetizione di un identificatore nei passaggi successivi non dimostra, di per sé, che l'autorità esercitata appartenga alla stessa linea causale di esecuzione. Una linea ha inizio quando un'entità dotata di permessi esprime un intento entro i propri privilegi, creando un contesto di autorità iniziale C_0 ; i passaggi successivi non devono necessariamente trasportare o reinterpretare l'identità dell'origine, ma devono conservare un contesto di autorità non espansivo e causalmente legato a essa.

Applicazione agli agenti di IA. Dal punto di vista della propagazione dell'autorità, gli agenti di IA sono un caso particolare di esecuzione distribuita autonoma; li menzioniamo esplicitamente perché in questo contesto la lacuna temporale è particolarmente marcata. Un agente esegue una catena di passi — pianificazione, invocazioni di strumenti, chiamate a valle — mentre dispone contemporaneamente di più fonti di autorità: le proprie credenziali di servizio, token utente delegati e ambiti specifici dei singoli strumenti. La classica designazione del nome di un file locale riappare come parametro influenzato dall'agente: un identificatore di tenant, documento o ruolo, un messaggio in coda o la destinazione di una richiesta. Quando il chiamante influenza

tale destinazione e l'agente soddisfa la richiesta usando autorità proveniente da un contesto diverso, si ottiene il problema del delegato confuso in forma distribuita. PIC lega ogni invocazione di strumento e ogni richiesta a valle alla linea causale che l'ha determinata; pertanto, un'autorità applicata al di fuori dell'esecuzione che ha causato l'azione non costituisce una continuazione valida. Un agente incaricato di riassumere un documento contenente una prompt injection indiretta — per esempio istruzioni volte a cancellare file o a esfiltrare segreti — può tentare tali azioni, ma in PoC esse sono valide soltanto se il contesto di autorità iniziale contiene i privilegi corrispondenti: se autorizza soltanto (*summarize, d*), allora né (*delete, r*) né (*exfiltrate, r*) possono costituire continuazioni valide.

Lavori correlati

Il problema del delegato confuso è stato introdotto da Hardy [1]. Il controllo degli accessi basato su capability [2] e l'analisi del delegato confuso nel contesto delle capability [3] affrontano la dimensione spaziale dell'autorità: se un attore detenga autorità su un oggetto. Questo lavoro isola la dimensione temporale, a essa ortogonale: se l'autorità impiegata appartenga all'esecuzione che ha causato l'azione. Consideriamo il delegato confuso non come lo specifico scenario del compilatore di sistema operativo nel quale fu descritto per la prima volta, ma come il disallineamento generale fra autorità e causalità esemplificato da quello scenario, e lo riformuliamo per l'esecuzione discreta in più passi: il contesto dei sistemi distribuiti, delle catene di servizi e degli agenti di IA, anziché di una singola invocazione locale. In questa formulazione, il delegato confuso è un'istanza di una classe di vulnerabilità nella propagazione dell'autorità accomunate dallo stesso disallineamento; con la Prova di Continuità, tale disallineamento non può presentarsi come comportamento valido del modello, poiché un privilegio assente dal contesto di autorità iniziale non può essere esercitato successivamente nella medesima linea causale. Il lavoro confronta la propagazione dell'autorità basata sul possesso e quella basata sulla continuità della provenienza esattamente lungo questo asse.

Il precedente più vicino è il controllo degli accessi basato sulla storia [4], nel quale l'autorità disponibile a una computazione dipende dalla storia di quanto è stato eseguito. Condividiamo il presupposto che l'autorità sia funzione dell'esecuzione, anziché del possesso statico. Diversamente da una verifica retrospettiva sui chiamanti precedenti (come nell'ispezione dello stack), PoC è una disciplina di propagazione in avanti: ogni passaggio deve

essere legato causalmente al predecessore e non deve espandere l'autorità ricevuta, anche fra passi asincroni e oltre i confini dei sistemi. Il contributo qui proposto è ortogonale e più specifico: formuliamo la dipendenza come una *dimensione* $\mathcal{L}$ ortogonale al privilegio (o, r) , dimostriamo che il possesso (ogni policy invariante rispetto a $\mathcal{L}$) non può distinguere le occorrenze e mostriamo che il delegato confuso è una collisione nella proiezione che dimentica $\mathcal{L}$. Ne segue che le mitigazioni basate sul possesso capaci di distinguere tali casi devono divenire sensibili alla linea causale e, quindi, alla continuità. La letteratura sulle capability di oggetto [5] analizza il delegato confuso come separazione fra designazione e autorità, elementi che le capability riuniscono: questa è la soluzione spaziale. Il nostro risultato aggiunge l'asse temporale: nell'astrazione del lavoro, una capability trattata soltanto come artefatto portabile e posseduto, senza vincoli all'occorrenza o alla linea causale, ricade nella proiezione invariante rispetto alla linea e non può distinguere due occorrenze che differiscono soltanto per essa.

Il controllo degli accessi basato sulla provenienza [19] usa la provenienza registrata degli oggetti come dato in ingresso alle decisioni di autorizzazione; PIC rende invece la linea causale dell'*esecuzione* stessa il vettore di un contesto di autorità non espansivo. Il controllo decentralizzato dei flussi di informazione [20] propaga etichette sui dati nel rispetto di una restrizione monotona; la Prova di Continuità propaga l'autorità lungo una linea causale di esecuzione secondo la medesima disciplina di monotonia, ma cambia l'oggetto propagato: l'autorità anziché le etichette dei flussi di informazione.

Token al portatore, macaroon e vincoli di prova del possesso. OAuth 2.0 comprende l'uso di token al portatore come schema comune di autorizzazione [6, 7]. Un token al portatore impiegato senza un vincolo esplicito al contesto o alla linea causale consente una decisione che legge il token e (o, r) , ma non la linea causale del suo uso: è invariante rispetto a essa e ricade quindi nel primo teorema. I macaroon [8] aggiungono *restrizioni* che possono soltanto attenuarsi e concatenarsi crittograficamente — esattamente la nostra attenuazione monotona $C_{i+1} \preceq C_i$ —, ma un macaroon resta una credenziale al portatore: in assenza di una restrizione che lo legghi al contesto della richiesta, può essere presentato fuori dalla linea che lo ha prodotto. DPoP [9] e Token Binding [10] vincolano un token a una chiave o a un canale di trasporto; nei termini del Teorema 4, tali vincoli fanno sì che la decisione legga un ulteriore contesto dell'occorrenza e sono quindi localmente sensibili alla continuità. Ciascun meccanismo riduce l'uso improprio fra contesti nella misura in cui una restrizione o un vincolo induce la decisione a leggere

ℓ ; nessuno, da solo, stabilisce una Prova di Continuità globale, poiché la decisione non è ancora tenuta a leggere l'intera linea causale dall'origine lungo una catena multipassaggio. PIC rende globale ed esplicita tale dimensione.

Il modello PIC è sviluppato indipendentemente in [17], dove la Provenance Identity Continuity viene introdotta come modello generale per i sistemi di esecuzione distribuiti. Il presente lavoro estende una versione precedente [18], affinandola mediante la distinzione fra relazione e continuità e l'analisi spaziale e temporale dell'autorità.

9 Limiti e sviluppi futuri

Il lavoro presenta un modello lineare minimale della Prova di Continuità. Molte delle direzioni indicate di seguito non sono difetti del modello, ma proprietà da sviluppare entro il medesimo principio di continuità; distinguiamo pertanto i limiti effettivi dell'ambito dalle estensioni naturali. In primo luogo, la revoca non viene trattata come operazione retroattiva su una linea causale già emessa: un sistema concreto deve stabilire se essa blocchi soltanto le transizioni future, invalidi le continuazioni ancora disponibili oppure determini l'annullamento dell'intera linea. In secondo luogo, la composizione: il modello è lineare, mentre biforcazioni, ricongiungimenti, fan-out, fan-in ed esecuzioni a forma di DAG costituiscono una naturale generalizzazione di $\mathcal{L}$ a un ordine parziale — un'estensione del principio di continuità a linee composte, non un suo limite. In terzo luogo, la costruzione di *PoR* è astratta: la sua realizzazione concreta è un problema implementativo distinto dal modello di continuità. L'evidenza di relazione richiesta può essere ricavata da meccanismi quali ambienti di esecuzione fidati o credenziali verificabili, con proprietà di sicurezza e prestazioni dipendenti dall'implementazione; il modello è neutrale rispetto alla scelta. In quarto luogo, la validazione della linea causale è un problema implementativo, non un limite del modello: le realizzazioni concrete non devono necessariamente rivalidare l'intera storia a ogni passaggio e possono usare checkpoint, prove sintetiche, codifiche compatte o costruzioni basate su accumulatori per mantenere limitato il costo della validazione. In quinto luogo, l'origine della linea causale: il modello vincola l'autorità *entro* una linea, ma non disciplina di per sé quando un esecutore possa legittimamente aprirne una nuova, radicata nei propri privilegi. Un esecutore influenzato da un avversario potrebbe classificare erroneamente come autonoma un'azione causata da una richiesta, compiendola fuori dalla catena della vittima senza espandere alcun C_i . Distinguere le azioni causate da richieste da quelle che hanno origine nell'esecutore compete alla costru-

zione di *PoR* e all'architettura di applicazione, ed è lasciato a lavori futuri. Infine, i sistemi eterogenei richiedono traduzioni di policy $\mathcal{T}$ fra vocabolari locali; ne rinviando a lavori futuri la sintesi, la verifica e la governance.

10 Conclusioni

Abbiamo introdotto la Prova di Continuità come modello temporale minimale per la propagazione dell'autorità. La distinzione fondamentale è fra possesso, che dimostra il controllo di un artefatto di autorità; relazione, che dimostra il legame causale fra passi di esecuzione adiacenti; e continuità, che dimostra come l'autorità si sia propagata senza espandersi lungo l'intera linea causale. Il risultato principale è che il possesso invariante rispetto alla linea non può distinguere occorrenze che condividono il medesimo privilegio ma discendono da cause diverse; viceversa, con la Prova di Continuità, un privilegio assente dal contesto di autorità iniziale non può comparire in seguito nella medesima catena di esecuzione valida. La condizione del delegato confuso non viene quindi accettata come comportamento valido del modello: essa è precisamente il disallineamento fra autorità e causalità escluso dalla continuità. Più in generale, PoC riformula la propagazione dell'autorità come proprietà temporale di una linea causale di esecuzione, anziché come mera proprietà statica del possesso di un artefatto.

Nota sul nome PIC. In Provenance Identity Continuity, il termine Identity indica l'identità di sicurezza o di esecuzione dell'entità dotata di permessi dalla quale l'autorità viene creata o ancorata all'origine. Non significa che l'identità venga propagata come primitiva di autorizzazione a ogni passaggio. Il modello sposta invece l'onere dell'autorizzazione dalla reiterata interpretazione dell'identità alla verifica della continuità dell'esecuzione: una linea causale che trasporta un contesto di autorità non espansivo. Provenance indica l'origine e la linea causale dell'esecuzione; Identity, l'origine autenticata e dotata di permessi; Continuity, la conservazione senza espansione dell'autorità nei passaggi successivi. Identità e identificatori restano pertanto essenziali per autenticazione, presentazione delle credenziali, audit, attribuzione di responsabilità e inizializzazione dell'origine; PoC riguarda la continuità dell'autorizzazione che deve valere dopo che tale origine è stata stabilita.

Ringraziamenti

Questo lavoro fa seguito a una versione precedente del modello PIC, resa disponibile su Zenodo nel dicembre 2025 [17]. L'autore ringrazia Alan H. Karp, la cui esperienza è stata preziosa per mettere alla prova i concetti di Prova di Continuità e Prova di Relazione. I concetti qui presentati sono propri dell'autore e il ringraziamento non implica l'approvazione del lavoro da parte sua. L'autore si è inoltre avvalso di strumenti automatizzati di assistenza linguistica per l'affinamento editoriale, la formattazione LaTeX e il perfezionamento della notazione. Tutti i contributi concettuali, i modelli e le dimostrazioni sono opera dell'autore.

Riferimenti bibliografici

- [1] N. Hardy. The Confused Deputy: Or Why Capabilities Might Have Been Invented. *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.
- [2] J. B. Dennis and E. C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [3] M. S. Miller, K.-P. Yee, and J. S. Shapiro. Capability Myths Demolished. Technical Report SRL2003-02, Systems Research Laboratory, Johns Hopkins University, 2003.
- [4] M. Abadi and C. Fournet. Access Control Based on Execution History. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2003.
- [5] M. S. Miller. Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, Baltimore, MD, USA, 2006.
- [6] D. Hardt, editor. The OAuth 2.0 Authorization Framework. RFC 6749, Internet Engineering Task Force, October 2012.
- [7] M. Jones and D. Hardt. The OAuth 2.0 Authorization Framework: Bearer Token Usage. RFC 6750, Internet Engineering Task Force, October 2012.

- [8] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.
- [9] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. B. Jones, and D. Waite. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449, Internet Engineering Task Force, September 2023.
- [10] A. Popov, M. Nyström, D. Balfanz, and J. Hodges. The Token Binding Protocol Version 1.0. RFC 8471, Internet Engineering Task Force, October 2018.
- [11] IETF WIMSE Working Group. Workload Identity in Multi-System Environments (WIMSE). Internet Engineering Task Force, 2024. <https://datatracker.ietf.org/wg/wimse/>.
- [12] SPIFFE Project. Secure Production Identity Framework for Everyone (SPIFFE) Specification. Cloud Native Computing Foundation, 2017. <https://github.com/spiffe/spiffe>.
- [13] OpenID Foundation. OpenID Connect Core 1.0. 2014. https://openid.net/specs/openid-connect-core-1_0.html.
- [14] OpenID Foundation. OpenID for Verifiable Credentials (OpenID4VC). Specification, 2024. <https://openid.net/sg/openid4vc/>.
- [15] World Wide Web Consortium (W3C). Decentralized Identifiers (DIDs) v1.0. W3C Recommendation, 2022. <https://www.w3.org/TR/did-core/>.
- [16] World Wide Web Consortium (W3C). Verifiable Credentials Data Model v1.1. W3C Recommendation, 2022. <https://www.w3.org/TR/vc-data-model/>.
- [17] N. Gallo. PIC Model — Provenance Identity Continuity for Distributed Execution Systems. Zenodo, 2025. <https://zenodo.org/records/17777421>.
- [18] N. Gallo. Authority Propagation Models: PoP vs PoC and the Confused Deputy Problem. Zenodo, 2025. <https://doi.org/10.5281/zenodo.17833000>.

- [19] J. Park, D. Nguyen, and R. Sandhu. A Provenance-Based Access Control Model. In *Proceedings of the International Conference on Privacy, Security and Trust (PST)*, 2012.
- [20] A. C. Myers and B. Liskov. A Decentralized Model for Information Flow Control. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 1997.